

Slides Activity 3:

Study planning

Table of contents

Using simulations to plan a study	2
Before you begin	2
Step 1. Outline the specific aims.	2
Step 2. Determine the appropriate data generating mechanism.	3
Helper functions	3
Function for DGM	3
Set the seed	4
The hypothesized effect	5
Step 3. Define the estimand of the analysis.	5
Step 4. Identify the methods to be implemented in the study.	5
Step 5. Specify the performance measures.	5
Simulating across a range of sample sizes	6
Function to simulate one sample-size scenario	6
Run this across a grid of candidate sample sizes	7
How does N affect precision (ASE, sandwich ASE, and ESE)?	8
How does N affect power?	9
What effect could I detect with the data I have?	10
Function to simulate one effect-size scenario	11
Translating a hypothesized risk difference into the DGM	11
Run this across a grid of hypothesized effect sizes, at a couple of fixed sample sizes	12
Plot: power vs. hypothesized effect size, by sample size	13
The minimum detectable effect size, at each N	13

Using simulations to plan a study

If we can posit a plausible data generating mechanism (DGM) for our study, we can use it to ask design questions.

When designing a prospective study, we typically ask “given the effect I hypothesize, how large a sample do I need?”

But when we’re using secondary data — a claims database, a disease registry — the sample size is set by the data.

In this activity we revisit the prenatal vitamins / preterm birth / smoking example from Activities 1–2.

Before you begin

Install the required packages.

```
library(dplyr)
library(tidyr)
library(purrr)
library(ggplot2)
library(kableExtra)
library(scales)
library(sandwich)
```

Step 1. Outline the specific aims.

Context: We are planning a study of the effect of prenatal vitamin supplementation on the risk of preterm birth, worried about confounding by maternal smoking (unmeasured in our planned data source). Before we commit to a data source and sample size, we want to understand how sample size shapes what our study can tell us.

Simulation Objectives:

1. Assuming our hypothesized risk difference is correct, quantify how precision (ASE from both model-based and sandwich standard errors, and ESE) and the proportion of studies whose confidence interval would show a clear difference (power) change across a range of candidate sample sizes.
2. Turning that question around: at a couple of fixed, realistic sample sizes (as we might encounter with an existing registry or claims database), quantify the smallest effect size we could realistically detect.

Step 2. Determine the appropriate data generating mechanism.

Helper functions

First, create the “helper” functions we will use throughout our simulation.

```
# The expit function is the inverse of the log-odds. It takes in a log-odds (x),
# applies the inverse-logit function and returns a probability
expit <- function(x) {
  return(exp(x) / (1 + exp(x)))
}

# The balancing intercept function, so we can re-derive the intercept whenever we
# change other parameters
derive_intercept <- function(py, prob_prenatal, prob_smoke,
                             or_smoke, or_prenatal){

  # marginal probability of treatment
  prob_prenatal <- prob_prenatal[1] * (1-prob_smoke) + prob_prenatal[2] * prob_smoke

  -log(1/py - 1) - or_prenatal*prob_prenatal - or_smoke*prob_smoke
}
```

Function for DGM

This is the same DGM from Activities 1–2.

```
dgm <- function(simid = 0,
               n = 500,
               prob_smoke = 0.2,
               prob_prenatal = c(0.85, 0.65),
               or_prenatal = log(0.8),
               or_smoke = log(3),
               prob_preterm = 0.1){

  # Dynamically derive the intercept term
  y_intercept <- derive_intercept(py = prob_preterm,
                                  prob_prenatal = prob_prenatal,
                                  prob_smoke = prob_smoke,
                                  or_smoke = or_smoke,
                                  or_prenatal = or_prenatal)
```

```

dplyr::tibble(
  # Simulation number
  simid = simid,

  # Create n participants
  id = 1:n,

  # Randomly select smoking
  smoke = rbinom(n = n, size = 1, prob = prob_smoke),

  # Generate potential outcome under no treatment
  preterm_x0 = rbinom(n = n, size = 1,
    prob = expit(y_intercept + (or_smoke * smoke))),

  # Generate potential outcome under treatment
  preterm_x1 = rbinom(n = n, size = 1,
    prob = expit(y_intercept + (or_smoke * smoke) + or_prenatal)),

  # Generate observed treatment conditional on smoking
  p_prenatal = prob_prenatal[smoke + 1],
  p_prenatal_margin = prob_prenatal[1]*(1-prob_smoke) + prob_prenatal[2]*(prob_smoke),
  prenatal = rbinom(n = n, size = 1, prob = p_prenatal),

  # Assign actual outcome conditional on observed treatment
  preterm = preterm_x0*(1-prenatal) + preterm_x1*(prenatal),

  # Known IPT weight
  ipw = dplyr::if_else(
    prenatal == 1,
    p_prenatal_margin / p_prenatal,
    p_prenatal_margin / (1 - p_prenatal)
  )
)
}

```

Set the seed

ALWAYS before any random number generation!

```
set.seed(918273)
```

The hypothesized effect

Study planning mainly *before* we have data, so instead of an empirical “truth” recovered from a huge dataset, we plug in our best guess (from prior literature, pilot data, or clinical judgment) for each DGM parameter. We can still generate one very large sample under those hypothesized parameters just to confirm what marginal risk difference they imply.

```
truth <- dgm(n = 10000000)
true_rd <- mean(truth$preterm_x1) - mean(truth$preterm_x0)
true_rd
```

```
[1] -0.0218565
```

Notice this is a small effect — about a 2-percentage-point absolute reduction in preterm birth risk. Small, plausible drug effects like this are common in pharmacoepidemiology, and (as we’ll see) they demand large studies.

Step 3. Define the estimand of the analysis.

As in Activities 1–2, our estimand is the marginal risk difference, $E[Y^{x=1}] - E[Y^{x=0}]$: the risk of preterm birth had *everyone* in the target population taken prenatal vitamins vs. had *no one* taken them.

Step 4. Identify the methods to be implemented in the study.

We will estimate the IPT-weighted risk difference (known weights), exactly as in Activity 2. Alongside the model-based standard error, we’ll also compute the robust sandwich standard error, which accounts for the fact that we’re using weights.

Step 5. Specify the performance measures.

Precision. As in Activity 2, we’ll track a few measures of how precisely we can pin down the risk difference:

- **ASE, model-based:** the average, across simulated samples, of the model-based standard error reported by the regression — what our software *tells us* about precision in a typical single sample.
- **ASE, sandwich:** the same idea, but averaging the robust sandwich standard error instead.

- **ESE** (empirical standard error): the standard deviation of the estimated risk differences across simulated samples — how much the estimate *actually* bounces around from sample to sample. If an ASE is close to the ESE, that standard error is trustworthy.

Power. Here we define power simply in terms of the confidence interval: the proportion of simulated samples whose 95% confidence interval (built from the model-based standard error) for the risk difference excludes zero (the value that would mean “no difference”).

$$Power \approx P\left(0 \notin [\widehat{RD} - 1.96 \times \widehat{SE}(\widehat{RD}), \widehat{RD} + 1.96 \times \widehat{SE}(\widehat{RD})]\right)$$

In other words: if our hypothesized effect is real, in what proportion of studies of this size would the confidence interval actually show a clear difference, rather than being consistent with “no effect”?

Simulating across a range of sample sizes

Function to simulate one sample-size scenario

This function draws `nsim` replicate samples at a given `n`, fits the IPTW model to each, and returns the estimated risk difference, its model-based and sandwich standard errors, and whether the model-based 95% CI excludes zero — for every replicate.

```
run_scenario <- function(n, nsim = 300) {
  purrr::map_dfr(1:nsim, function(i) {
    dat <- dgm(simid = i, n = n)

    unadj <- glm(preterm ~ prenatal, data = dat, family = binomial(link = "identity"))
    adj_known <- glm(preterm ~ prenatal, data = dat,
                     weights = ipw, family = quasibinomial(link = "identity"))

    dplyr::tibble(
      estimator = c("Unadjusted", "Adjusted (Naive SE)", "Adjusted (Robust SE)"),

      # extract risk different from the models
      rd = c(coef(unadj)[2], coef(adj_known)[2], coef(adj_known)[2]),

      # extract the standard errors from the models
      se = c(sqrt(diag(vcov(unadj)))[2],
             sqrt(diag(vcov(adj_known)))[2],
             sqrt(diag(sandwich::vcovHC(adj_known)))[2])
    )
  })
}
```

```

}) |>
dplyr::mutate(
  lcl = rd - 1.96*se,
  ucl = rd + 1.96*se,
  # Does this replicate's 95% CI exclude zero?
  excludes_zero = (lcl > 0) | (ucl < 0),
  # Does this replicate's CI include the truth?
  includes_truth = lcl <= true_rd & true_rd <= ucl
)
}

```

Run this across a grid of candidate sample sizes

```

n_grid <- c(500, 1000, 2000, 5000, 10000, 20000, 40000)

results_by_n <- purrr::map_dfr(n_grid, function(n) {
  reps <- run_scenario(n, nsim = 300)
  reps |>
    dplyr::summarize(
      n = n,
      power = mean(excludes_zero),
      ase = sqrt(mean(se^2)),
      ese = sd(rd),
      coverage = mean(includes_truth),
      .by = estimator
    )
})

results_by_n |>
  dplyr::arrange(estimator, n) |>
  kableExtra::kable(col.names = c("Estimator", "N", "Power", "ASE", "ESE", "95% confidence interval coverage"),
    digits = c(0, 3, 4, 4, 4, 3))

```

Estimator	N	Power	ASE	ESE	95% confidence interval coverage
Adjusted (Naive SE)	500	0.2167	0.0286	0.0370	0.850
Adjusted (Naive SE)	1000	0.2500	0.0201	0.0271	0.850
Adjusted (Naive SE)	2000	0.3600	0.0143	0.0193	0.877
Adjusted (Naive SE)	5000	0.6200	0.0090	0.0117	0.873
Adjusted (Naive SE)	10000	0.9067	0.0064	0.0080	0.877

Estimator	N	Power	ASE	ESE	95% confidence interval coverage
Adjusted (Naive SE)	20000	0.9867	0.0045	0.0062	0.840
Adjusted (Naive SE)	40000	1.0000	0.0032	0.0043	0.843
Adjusted (Robust SE)	500	0.0733	0.0381	0.0370	0.960
Adjusted (Robust SE)	1000	0.1167	0.0267	0.0271	0.933
Adjusted (Robust SE)	2000	0.1667	0.0189	0.0193	0.940
Adjusted (Robust SE)	5000	0.4433	0.0119	0.0117	0.933
Adjusted (Robust SE)	10000	0.7467	0.0084	0.0080	0.957
Adjusted (Robust SE)	20000	0.9700	0.0059	0.0062	0.957
Adjusted (Robust SE)	40000	1.0000	0.0042	0.0043	0.933
Unadjusted	500	0.2367	0.0398	0.0389	0.900
Unadjusted	1000	0.4633	0.0280	0.0284	0.830
Unadjusted	2000	0.7500	0.0198	0.0206	0.717
Unadjusted	5000	0.9867	0.0125	0.0124	0.330
Unadjusted	10000	1.0000	0.0089	0.0085	0.057
Unadjusted	20000	1.0000	0.0063	0.0066	0.003
Unadjusted	40000	1.0000	0.0044	0.0045	0.000

How does N affect precision (ASE, sandwich ASE, and ESE)?

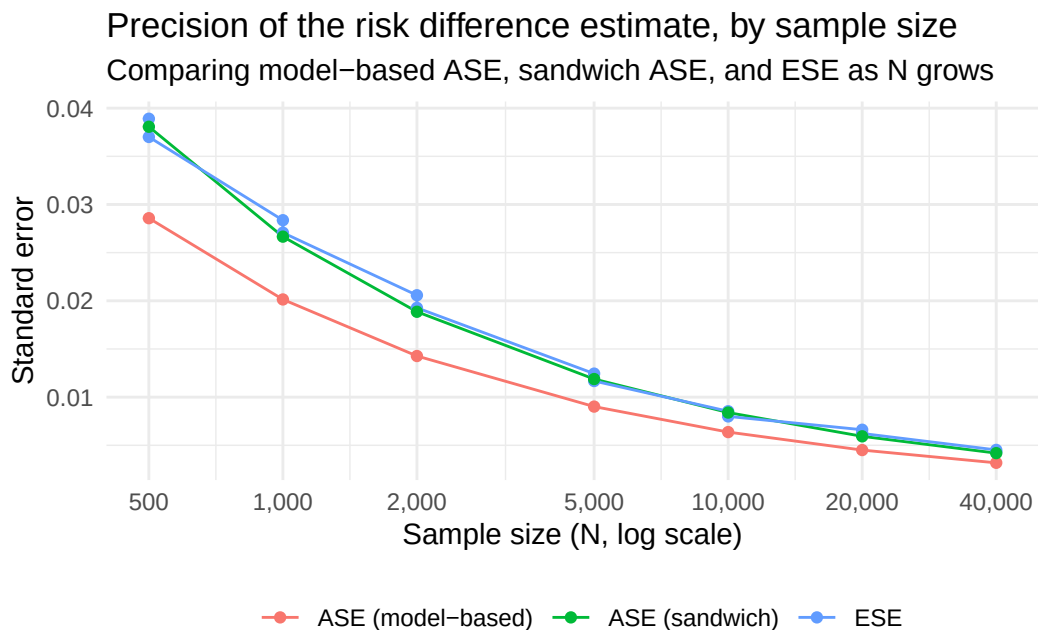
```
results_by_n |>
  tidyr::pivot_longer(cols = c(ase, ese), names_to = "measure", values_to = "value") |>
  dplyr::mutate(
    measure = dplyr::case_when(
      measure == "ese" ~ "ESE",
      measure == "ase" & estimator == "Adjusted (Naive SE)" ~ "ASE (model-based)",
      measure == "ase" & estimator == "Adjusted (Robust SE)" ~ "ASE (sandwich)") |>
  distinct(n, value, measure) |>
  dplyr::filter(!is.na(measure)) |>
  ggplot(aes(x = n, y = value, color = measure)) +
  geom_line() +
  geom_point() +
  scale_x_log10(breaks = n_grid, labels = scales::label_comma()) +
```



```

theme_minimal() +
labs(
  x = "Sample size (N, log scale)",
  y = "Standard error",
  color = "",
  title = "Precision of the risk difference estimate, by sample size",
  subtitle = "Comparing model-based ASE, sandwich ASE, and ESE as N grows"
) +
theme(legend.position = "bottom")

```



How does N affect power?

```

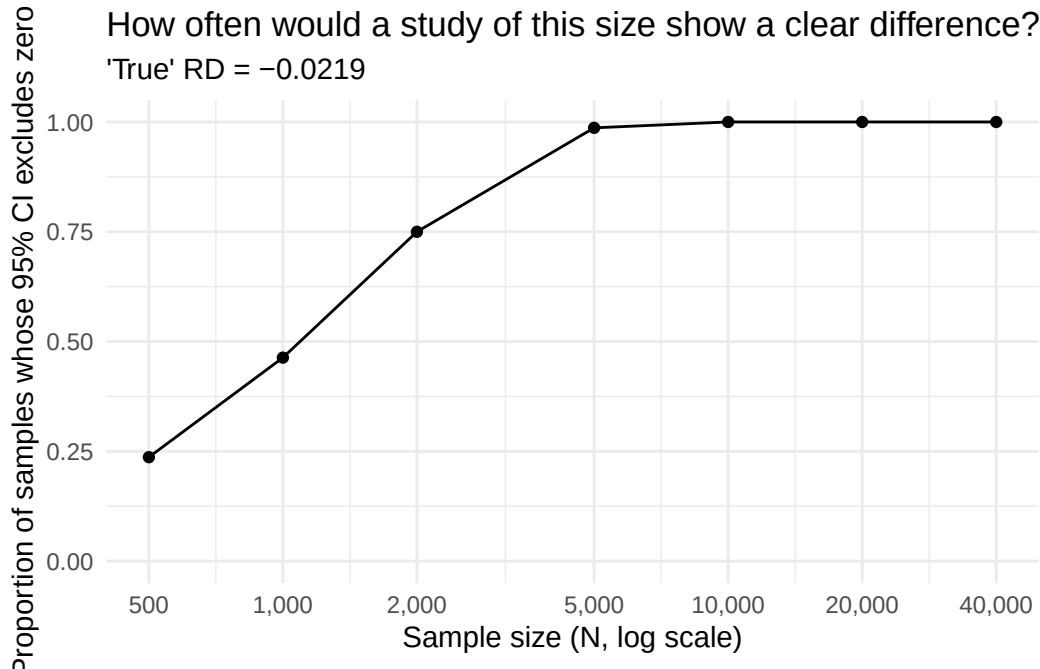
results_by_n |>
  dplyr::filter(estimator == "Unadjusted") |>
  ggplot(aes(x = n, y = power)) +
  geom_line() +
  geom_point() +
  scale_x_log10(breaks = n_grid, labels = scales::label_comma()) +
  scale_y_continuous(limits = c(0, 1)) +
  theme_minimal() +
  labs(
    x = "Sample size (N, log scale)",

```

```

y = "Proportion of samples whose 95% CI excludes zero",
title = "How often would a study of this size show a clear difference?",
subtitle = paste0("'True' RD = ", round(true_rd, 4))
)

```



What effect could I detect with the data I have?

So far we've asked: "given a hypothesized effect, how big a sample do I need?" But in pharmacoepidemiology we're typically using an existing data source (an insurance claims database, a disease registry). The useful planning question becomes: "given the N in this data source (and my design choices), what's the smallest effect I could realistically detect?"

We can answer this with the exact same simulation machinery, just swapping which quantity is fixed and which one varies.

Instead of looping over N at one hypothesized effect, we now fix N and loop over a grid of hypothesized effect sizes. We can also change the confounding parameters to see how that impacts our precision.

Function to simulate one effect-size scenario

```
run_effect_scenario <- function(n, or_prenatal, nsim = 250) {  
  purrr::map_dfr(1:nsim, function(i) {  
    dat <- dgm(simid = i, n = n, or_prenatal = or_prenatal)  
  
    fit <- glm(preterm ~ prenatal, data = dat,  
              weights = ipw, family = quasibinomial(link = "identity"))  
  
    dplyr::tibble(  
      rd = coef(fit)[2],  
      se = sqrt(diag(sandwich::vcovHC(fit)))[2]  
    )  
  }) |>  
  dplyr::mutate(  
    ci_width = (rd + 1.96*se) - (rd - 1.96*se),  
    excludes_zero = (rd - 1.96*se > 0) | (rd + 1.96*se < 0)  
  )  
}
```

We use the sandwich standard error directly here, since we know it more accurately estimates the empirical standard error.

Translating a hypothesized risk difference into the DGM

Our DGM generates the treatment effect on the log-odds scale (`or_prenatal`), but the effect we actually want to hypothesize about is the marginal risk difference. So first we write down, analytically, the exact marginal risk difference implied by a given `or_prenatal` (no simulation noise, since `smoke` only takes two values), then invert that numerically to find the `or_prenatal` that produces whatever risk difference we want to hypothesize.

```
implied_rd <- function(or_prenatal, or_smoke = log(3), prob_smoke = 0.2,  
                      prob_prenatal = c(0.85, 0.65), prob_preterm = 0.1) {  
  y_intercept <- derive_intercept(py = prob_preterm, prob_prenatal = prob_prenatal,  
                                  prob_smoke = prob_smoke, or_smoke = or_smoke,  
                                  or_prenatal = or_prenatal)  
  p_x0 <- (1 - prob_smoke) * expit(y_intercept) +  
    prob_smoke * expit(y_intercept + or_smoke)  
  p_x1 <- (1 - prob_smoke) * expit(y_intercept + or_prenatal) +  
    prob_smoke * expit(y_intercept + or_smoke + or_prenatal)
```

```

  p_x1 - p_x0
}

or_prenatal_for_rd <- function(target_rd) {
  uniroot(function(lor) implied_rd(lor) - target_rd, interval = log(c(0.001, 10)))$root
}

```

Run this across a grid of hypothesized effect sizes, at a couple of fixed sample sizes

```

# Two data sources we don't get to redesign: a modest single-site registry,
# and a much larger insurance claims database.
rd_grid <- c(-0.01, -0.02, -0.03, -0.05, -0.07, -0.10)
fixed_n <- c(1000, 10000)
detectable_effect <- purrr::map_dfr(fixed_n, function(n) {
  purrr::map_dfr(rd_grid, function(target_rd) {
    reps <- run_effect_scenario(n, or_prenatal = or_prenatal_for_rd(target_rd), nsim = 250)
    dplyr::tibble(n = n, rd = target_rd, power = mean(reps$excludes_zero))
  })
})

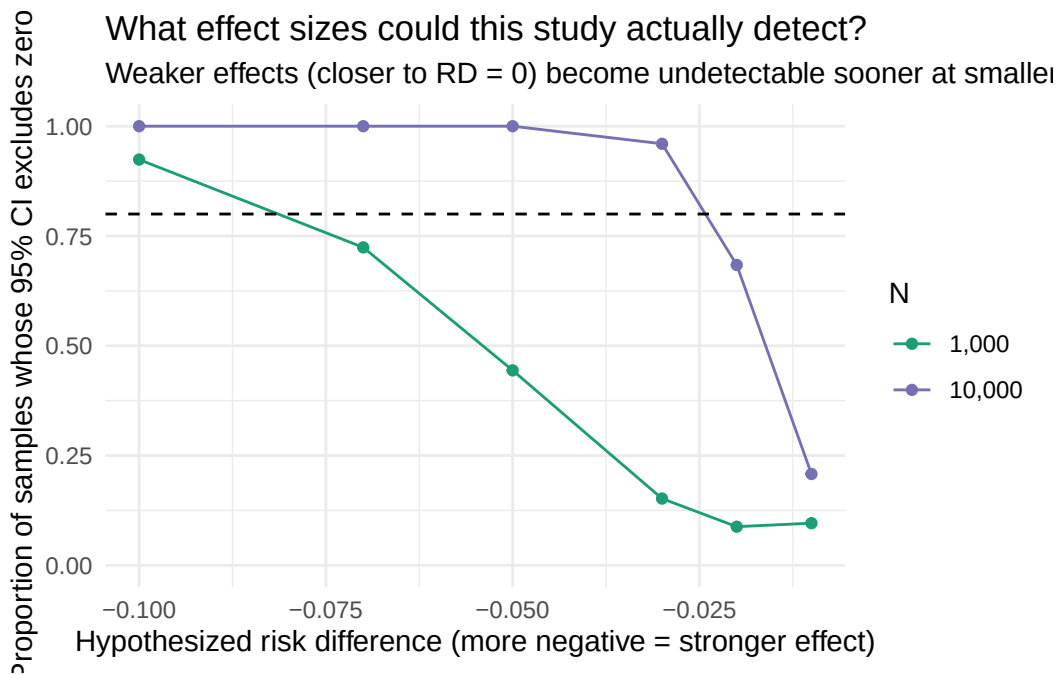
detectable_effect |>
  kableExtra::kable(col.names = c("N", "Hypothesized risk difference", "Power"), digits = 3)

```

N	Hypothesized risk difference	Power
1000	-0.01	0.096
1000	-0.02	0.088
1000	-0.03	0.152
1000	-0.05	0.444
1000	-0.07	0.724
1000	-0.10	0.924
10000	-0.01	0.208
10000	-0.02	0.684
10000	-0.03	0.960
10000	-0.05	1.000
10000	-0.07	1.000
10000	-0.10	1.000

Plot: power vs. hypothesized effect size, by sample size

```
ggplot(detectable_effect, aes(x = rd, y = power, color = factor(n))) +
  geom_line() +
  geom_point() +
  geom_hline(yintercept = 0.8, linetype = "dashed") +
  scale_y_continuous(limits = c(0, 1)) +
  scale_color_manual(values = c("#1b9e77", "#7570b3"), labels = scales::comma(fixed_n)) +
  theme_minimal() +
  labs(
    x = "Hypothesized risk difference (more negative = stronger effect)",
    y = "Proportion of samples whose 95% CI excludes zero",
    color = "N",
    title = "What effect sizes could this study actually detect?",
    subtitle = "Weaker effects (closer to RD = 0) become undetectable sooner at smaller N"
  )
```



The minimum detectable effect size, at each N

```

mdes <- detectable_effect |>
  dplyr::arrange(n, power) |>
  dplyr::summarize(
    min_detectable_rd = approx(x = power, y = rd, xout = 0.8)$y,
    .by = n
  )

mdes |>
  kableExtra::kable(col.names = c("N", "Smallest detectable risk difference (80% power)"), d

```

N	Smallest detectable risk difference (80% power)
1000	-0.081
10000	-0.024